

Neural Networks in Three Minutes

Neurons, layers, weights, and learning · video 2:00

This lesson has one big idea: a neural network is **just a function**, built from one very simple part repeated many times. A **neuron** weighs its inputs, adds a bias and bends the result; neurons side by side make a **layer**, and stacked layers make a **network**. Its numbers, the **parameters**, are not written by anyone: they are learned from data.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A neuron: weigh, add, bend	▶ 0:08-0:42	4	☐
2. A layer is one matrix multiplication	▶ 0:42-0:54	3	☐
3. Stacking layers	▶ 0:54-1:03	4	☐
4. Parameters: the numbers that learn	▶ 1:03-1:15	4	☐
5. Inference and training	▶ 1:15-1:30	4	☐
6. We choose the shape, data does the rest	▶ 1:30-1:38	3	☐
7. Neurons and layers in code	▶ 1:38-1:45	2	☐

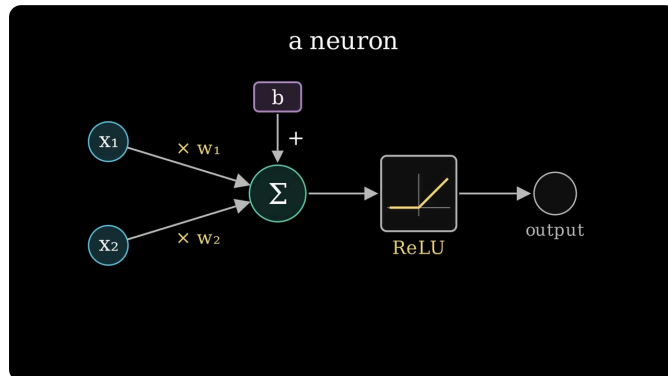
Before you start: this lesson builds on F02 (The Dot Product), F03 (Matrices) and F04 (Straight Lines and Bends, where ReLU was introduced); the code uses NumPy's @ from F11. Neural networks return in episode 8 (The MLP: Where Facts Live), episode 11 (Training: Learning from Mistakes) and episode 12 (Build a Tiny GPT).

CONCEPT 1 OF 7

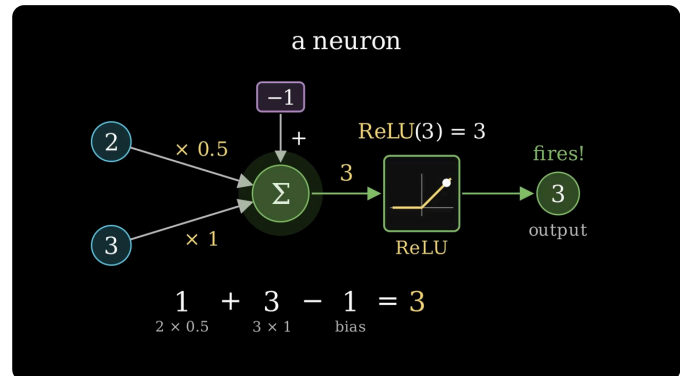
A neuron: weigh, add, bend

▶ Video 0:08-0:42

Strip away the hype and a neural network is a **function** built from simple parts. The basic part is a **neuron**: it multiplies each input by a **weight**, adds the results up with a **bias**, and passes the sum through a **bend**, here **ReLU** (F04): positive numbers pass, negative numbers become 0.



▶ 0:27 Weigh each input, add them up with the bias (Σ), bend with ReLU.



▶ 0:41 The video's example: $1 + 3 - 1 = 3$, and $\text{ReLU}(3) = 3$.

Inputs 2 and 3, weights 0.5 and 1, bias -1: $2 \times 0.5 + 3 \times 1 - 1 = 3$. $\text{ReLU}(3) = 3$, so the neuron **fires** with value 3. Had the sum been negative, it would output 0 and stay silent.

KEY IDEA

A neuron computes **$\text{ReLU}(\mathbf{x} \cdot \mathbf{w} + \mathbf{b})$** : the dot product of the inputs with the weights, plus a bias, then the bend.

Check 1: A neuron: weigh, add, bend

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

Same neuron (weights 0.5 and 1, bias -1), inputs 4 and 1. What does it output?

1.2 CALCULATION

Same neuron, inputs 2 and -3. Compute the sum before the bend, and the output. Does the neuron fire?

1.3 PUT IN ORDER

Put a neuron's steps in order: *apply ReLU · add the bias · multiply each input by its weight · add up the products.*

1.4 MULTIPLE CHOICE

What does the bias do?

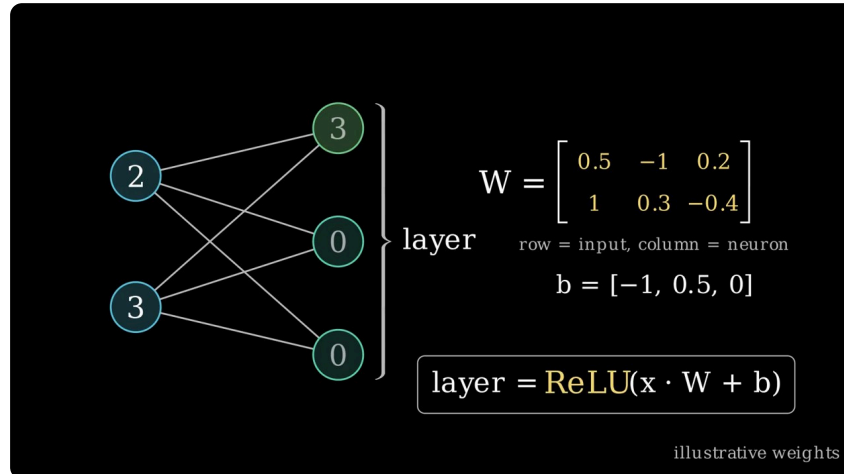
- A. It multiplies every input by the same number
- B. It adds a fixed number to the weighted sum, shifting it up or down before the bend
- C. It chooses which inputs the neuron ignores
- D. It is the bend that turns negative numbers into 0

CONCEPT 2 OF 7

A layer is one matrix multiplication

▶ Video 0:42-0:54

Put several neurons side by side, all reading the **same inputs**, and you have a **layer**. Each neuron keeps its own weights. Write them as the **columns** of a matrix **W** (one row per input, one column per neuron) and the biases as a vector **b**, and the whole layer is a single line: **ReLU(x · W + b)**.



▶ 0:53 Three neurons read the same inputs 2 and 3. Their weights form the matrix **W** (row = input, column = neuron): out comes [3, 0, 0].

In the video, column 1 of **W** is the neuron from concept 1, giving 3. Column 2 gives $2 \times (-1) + 3 \times 0.3 + 0.5 = -0.6$, column 3 gives $2 \times 0.2 + 3 \times (-0.4) + 0 = -0.8$. After ReLU: **[3, 0, 0]**.

KEY IDEA

A layer of neurons is **one matrix multiplication plus the bend**: $\text{ReLU}(x \cdot W + b)$. It turns the input vector into one number per neuron, all at once.

Check 2: A layer is one matrix multiplication

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Same layer ($W = [[0.5, -1, 0.2], [1, 0.3, -0.4]]$, $b = [-1, 0.5, 0]$), new input $x = [0, 2]$. Compute $x \cdot W + b$, then the layer's output.

.....

2.2 MULTIPLE CHOICE

A layer reads 4 inputs and has 10 neurons. What shape is **W**, and how many biases are there?

- A. 4×10 , and 10 biases
- B. 10×4 , and 4 biases
- C. 4×10 , and 4 biases
- D. 10×10 , and 10 biases

2.3 TRUE OR FALSE

"In $x \cdot W$, each *row* of **W** holds the weights of one neuron."

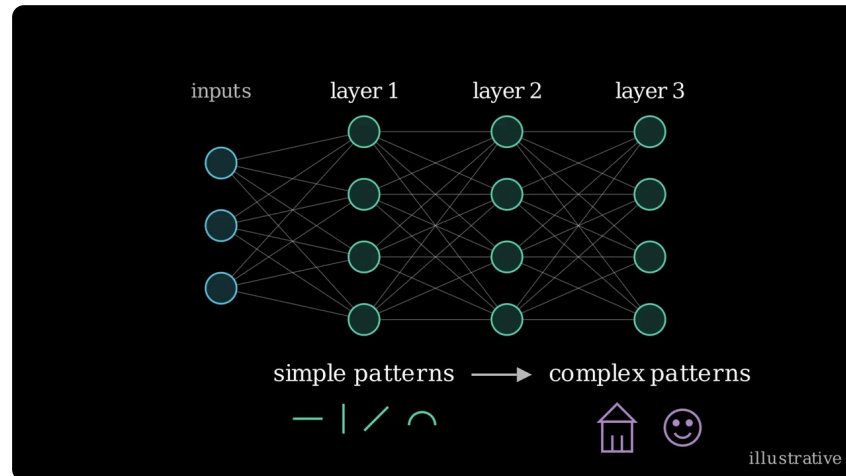
True ☐ False ☐

CONCEPT 3 OF 7

Stacking layers

▶ Video 0:54–1:03

Stack layers, and the **outputs of one layer become the inputs of the next**. Why? **Early layers find simple patterns; later layers combine them into more complex ones**, like strokes into shapes. The bend between layers makes this work: without it, two matrix multiplications in a row collapse into one (F04).



▶ 1:02 Each layer's outputs feed the next layer.

KEY IDEA

A network is a chain of layers: **the output of one layer is the input of the next**. Depth lets it build complex patterns out of simple ones.

Check 3: Stacking layers

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

The layer of concept 2 outputs $[3, 0, 0]$. How many inputs does the next layer read, and how many rows must its weight matrix have?

3.2 CALCULATION

The next layer has a single neuron with weights $[0.5, 4, -2]$ and bias 1. It reads $[3, 0, 0]$. What does it output?

3.3 MULTIPLE CHOICE

According to the video, how do the layers of a deep network typically divide the work?

- A. Early layers find complex patterns, later layers simplify them
- B. Early layers find simple patterns, later layers combine them into complex ones
- C. Every layer finds the same patterns, for safety
- D. Only the last layer finds patterns; the others pass the inputs through

3.4 TRUE OR FALSE

“Two layers with *no bend* between them can do more than one layer.”

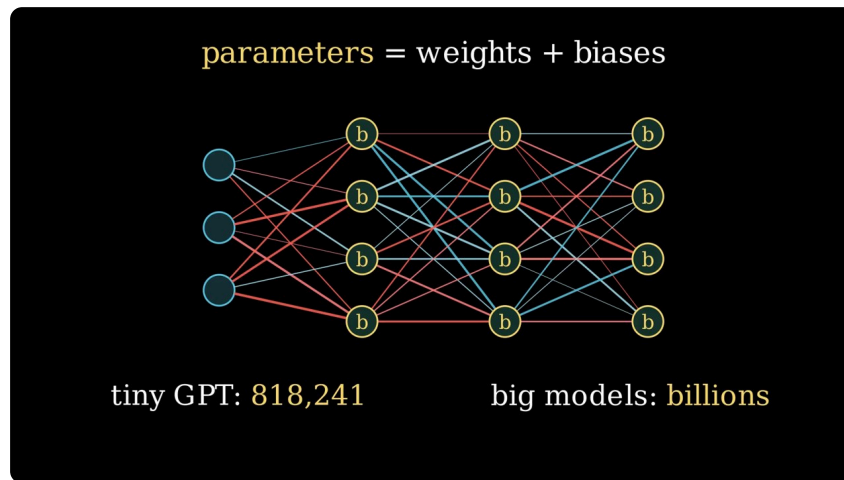
True ☐ False ☐

CONCEPT 4 OF 7

Parameters: the numbers that learn

▶ Video 1:03-1:15

The weights and biases together are the network's **parameters**. They are the **only thing that changes** when a network learns. The tiny GPT of episode 12 has **818,241**; big models have billions.



▶ 1:14 A weight on every edge, a bias (b) in every neuron.

A layer with n inputs and m neurons has **$n \times m$ weights** (one per connection) and **m biases** (one per neuron).

KEY IDEA

Parameters = weights + biases. One layer has inputs $\times$ neurons + neurons of them. Learning means changing these numbers, and nothing else.

Check 4: Parameters: the numbers that learn

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

How many parameters does the layer of concept 2 (2 inputs, 3 neurons) have?

4.2 CALCULATION

The network in the figure has 3 inputs and three layers of 4 neurons each. How many parameters does it have in total?

4.3 MULTIPLE CHOICE

Which of these is **not** a parameter of the network?

- A. The weight on the edge from input 1 to neuron 3
- B. The bias of neuron 2
- C. The input numbers, such as 2 and 3
- D. An entry of the matrix W

4.4 TRUE OR FALSE

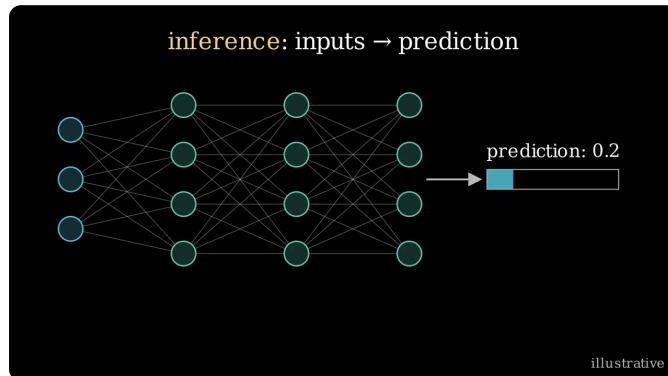
"When a network learns, it grows new neurons and connections."

True ☐ False ☐

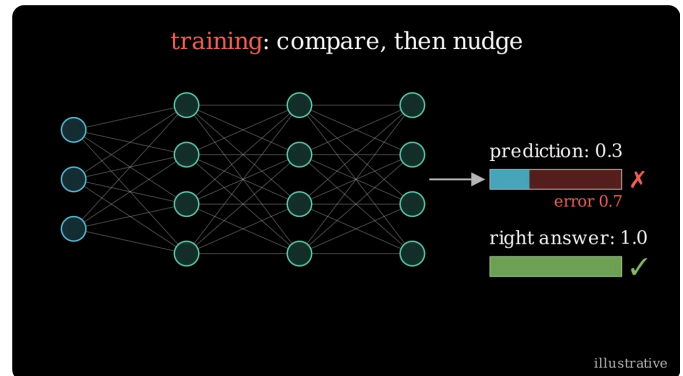
Inference and training

▶ Video 1:15-1:30

A network is used in two ways. **Inference** means running it: inputs flow forward and out comes a **prediction**; nothing inside changes. **Training** means adjusting it: **compare** the prediction with the right answer and **nudge every parameter** a little so the prediction gets better.



▶ 1:23 Inference: forward to a prediction.



▶ 1:29 Training: compare, then nudge.

In the video's illustration, the prediction 0.2 misses the right answer 1.0 by 0.8; one nudge later it is 0.3, an error of 0.7. Training repeats this many times (F10, F13 and episode 11 show how).

KEY IDEA

Inference: forward only, parameters fixed. **Training:** forward, compare with the right answer, nudge every parameter, and repeat.

Check 5: Inference and training

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 MULTIPLE CHOICE

You ask a finished chatbot a question and it answers. What is the network doing?

- A. Training
- B. Inference
- C. Both at once
- D. Neither

5.2 PUT IN ORDER

Put one training step in order: *nudge every parameter · compare the prediction with the right answer · run the inputs forward.*

5.3 CALCULATION

Continue the video's example: the prediction is 0.3 and the right answer 1.0. The next nudge moves the prediction to 0.45. What is the new error, and by how much did it shrink?

5.4 TRUE OR FALSE

"During inference, the parameters are nudged after every prediction."

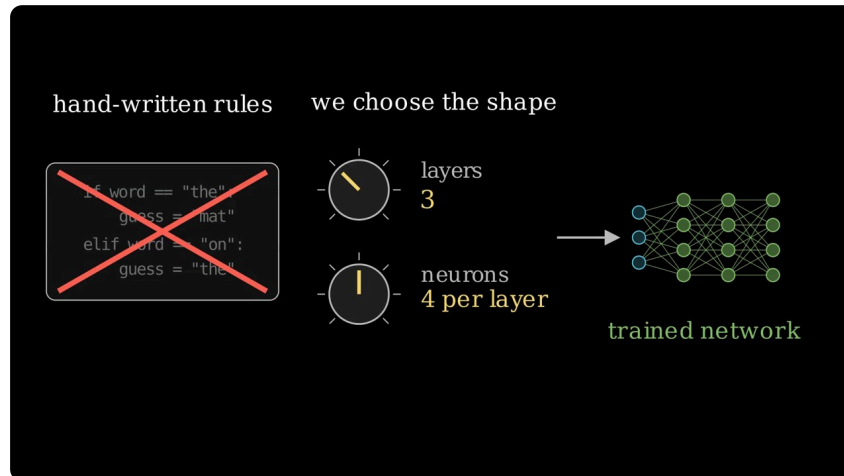
True ☐ False ☐

CONCEPT 6 OF 7

We choose the shape, data does the rest

▶ Video 1:30-1:38

Nobody writes the rules inside a neural network: there is no *if word == "the"* anywhere. People choose only the **shape**: how many layers, and how many neurons in each. The shape fixes how many parameters there are. Their **values** are set by training on data, which turns an untrained network into a trained one.



▶ 1:37 No hand-written rules. We set the shape (3 layers, 4 neurons each); training data turns the untrained network into a trained one.

KEY IDEA

People choose the **shape** (layers, neurons per layer). The **training data** chooses the values of the parameters.

Check 6: We choose the shape, data does the rest

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 MULTIPLE CHOICE

Which of these is chosen by a person, not learned from data?

- A. The weight between input 1 and neuron 3
- B. The number of layers
- C. The bias of neuron 2
- D. The output for a particular input

6.2 CALCULATION

You choose the shape: 2 inputs → a layer of 3 neurons → a layer of 1 neuron. How many parameters will training have to set?

6.3 TRUE OR FALSE

"To teach a network something new, an engineer writes a new rule into it."

True ☐ False ☐

CONCEPT 7 OF 7

Neurons and layers in code

▶ Video 1:38-1:45

In code, a neuron is a dot product, a bias and a ReLU. A layer does many neurons at once: the same line with a matrix `W` instead of a vector `w`, and `np.maximum`, which applies ReLU to every number of the result.

```
1 def neuron(x, w, b):
2     return max(0.0, x @ w + b)      # weighted sum, then ReLU
3
4 x = np.array([2.0, 3.0])
5 neuron(x, np.array([0.5, 1.0]), -1.0)  # 3.0
6
7 def layer(x, W, b):
8     return np.maximum(0, x @ W + b)  # many neurons at once
```

▶ 1:44 The code from the video: a neuron in one line, a whole layer in one line.

```
def neuron(x, w, b):
    return max(0.0, x @ w + b)      # weighted sum, then ReLU

x = np.array([2.0, 3.0])
neuron(x, np.array([0.5, 1.0]), -1.0)  # 3.0

def layer(x, W, b):
    return np.maximum(0, x @ W + b)  # many neurons at once
```

KEY IDEA

Neuron: `max(0.0, x @ w + b)`. Layer: `np.maximum(0, x @ W + b)`. A network is layers called one after another.

Check 7: Neurons and layers in code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

7.1 MULTIPLE CHOICE

Why does `layer` use `np.maximum(0, ...)` rather than Python's `max(0.0, ...)`?

- A. `np.maximum` is faster, but the result is the same
- B. `x @ W + b` is an array: `np.maximum` applies ReLU to each element, but `max` cannot compare a whole array with 0
- C. `np.maximum` picks the neuron with the largest output
- D. `np.maximum` also adds the bias

7.2 HANDS-ON CODE

Try it yourself. The code below has the two functions from the video, the video's first layer ($W1$, $b1$) and a second layer with two neurons ($W2$, $b2$). (a) What does `layer(x, W1, b1)` return? Check that `neuron(x, W1[:, 0], b1[0])` matches its first number. (b) Write `network(x)`, which feeds the output of layer 1 into layer 2. What does it return for x ? (c) What does it return for `np.array([0.0, 2.0])`? How many parameters does this network have?

```
import numpy as np

def neuron(x, w, b):
    return max(0.0, x @ w + b)          # weighted sum, then ReLU

def layer(x, W, b):
    return np.maximum(0, x @ W + b)     # many neurons at once

x = np.array([2.0, 3.0])
W1 = np.array([[0.5, -1.0, 0.2],
               [1.0, 0.3, -0.4]])
b1 = np.array([-1.0, 0.5, 0.0])
W2 = np.array([[ 1.0, -2.0],
               [ 0.5, 1.0],
               [-1.0, 3.0]])
b2 = np.array([0.5, 1.0])
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A neuron: weigh, add, bend • rewatch ▶ 0:08-0:42

1.1 2. $4 \times 0.5 + 1 \times 1 = 3$, then $3 - 1 = 2$. ReLU keeps positive numbers, so 2.

1.2 Sum -3, output 0: it does not fire. $2 \times 0.5 + (-3) \times 1 - 1 = 1 - 3 - 1 = -3$, and ReLU turns every negative number into 0.

1.3 multiply by weights → add up → add the bias → apply ReLU. Weigh, add, bend: the bias shifts the sum before the bend decides what comes out.

1.4 B. In the example the bias -1 turned 4 into 3. Weights scale the inputs (A); ReLU is the bend (D).

2. A layer is one matrix multiplication • rewatch ▶ 0:42-0:54

2.1 $x \cdot W + b = [1, 1.1, -0.8]$; output $[1, 1.1, 0]$. Column by column: $0 + 2 - 1 = 1$; $0 + 0.6 + 0.5 = 1.1$; $0 - 0.8 + 0 = -0.8$. ReLU zeroes only the negative one, so two neurons fire this time.

2.2 A. One row per input (4), one column per neuron (10), and one bias per neuron (10).

2.3 False. Each *column* is one neuron; each row belongs to one input. Column 1, $[0.5, 1]$, is the neuron from concept 1.

3. Stacking layers • rewatch ▶ 0:54-1:03

3.1 3 inputs, so 3 rows. One input per neuron of the previous layer, and W always has one row per input.

3.2 2.5. $3 \times 0.5 + 0 \times 4 + 0 \times (-2) + 1 = 2.5$, and ReLU keeps it. The silent neurons send 0, so their weights (4 and -2) make no difference for this input.

3.3 B. Simple pieces first (like strokes), then combinations of pieces (like shapes).

3.4 False. Without the bend, the two matrix multiplications collapse into one matrix (F04). The bend is what makes depth worthwhile.

4. Parameters: the numbers that learn • rewatch ▶ 1:03-1:15

4.1 9. $2 \times 3 = 6$ weights (the entries of W) plus 3 biases.

4.2 56. Layer 1: $3 \times 4 + 4 = 16$. Layers 2 and 3: $4 \times 4 + 4 = 20$ each. $16 + 20 + 20 = 56$.

4.3 C. Inputs are the data the network reads; they change with every example. Parameters belong to the network itself.

4.4 False. The shape stays fixed; only the values of the weights and biases change.

5. Inference and training • rewatch ▶ 1:15-1:30

5.1 B. It runs forward to make predictions; its parameters stay as they are.

5.2 run forward → compare → nudge. You need a prediction before you can compare it, and the comparison tells you how to nudge.

5.3 Error 0.55, which is 0.15 smaller. $1.0 - 0.45 = 0.55$, down from 0.7.

5.4 False. Nudging parameters is training. Inference only runs the network forward.

6. We choose the shape, data does the rest · rewatch ► 1:30-1:38

6.1 B. The number of layers is part of the shape. Weights and biases are learned, and the outputs follow from them.

6.2 13. First layer $2 \times 3 + 3 = 9$, second layer $3 \times 1 + 1 = 4$, and $9 + 4 = 13$.

6.3 False. Nobody writes rules inside. You train it on new data and the parameters change.

7. Neurons and layers in code · rewatch ► 1:38-1:45

7.1 B. `max(0.0, np.array([3, -0.6]))` raises a `ValueError` ("truth value of an array ... is ambiguous"). `np.maximum` works element by element.

7.2 (a) [3. 0. 0.], and neuron(...) = 3.0 · (b) [3.5 0.] · (c) [2.05 0.1], 17 parameters (a) The numbers of concept 2; column 0 of `W1` with bias `b1[0]` is the neuron of concept 1. (b) `def network(x): return layer(layer(x, W1, b1), W2, b2)`. The hidden output `[3, 0, 0]` gives $[3 \times 1 + 0.5, 3 \times (-2) + 1] = [3.5, -5]$, and `ReLU` makes it `[3.5, 0]`. (c) The hidden output is now `[1, 1.1, 0]` (question 2.1), so layer 2 computes $[1 + 0.55 + 0.5, -2 + 1.1 + 1] = [2.05, 0.1]$: both neurons fire. Parameters: $6 + 3 + 6 + 2 = 17$ (`W1.size + b1.size + W2.size + b2.size`).